Use Case 1: Start Focus Session (Focus Session UI)

When a user starts a focus session, the system validates inputs, creates a session, runs the timer, and upon completion, records the session and awards XP.

Algorithm 1: The startFocusSession() function — triggered when a user starts a focus session

Input:
userId — the ID of the logged-in user;
task — the selected Task object;
duration — session duration;
breakDuration — optional break time

Output:
A completed FocusSession record; updated XP; error message if invalid

Pseudocode:

Algorithm 1: The startFocusSession() function

Input: userId — ID of the user; task — selected Task object; duration — session duration

Output: A completed FocusSession record or error message

```
// 1. Validate inputs
if task is null then
    return error("Task is required")
if duration < 5 or duration > 90 then
    return error("Invalid session duration")

// 2. Create session
session ← new FocusSession(userId, task.taskId, duration)
session.startTime ← currentDateTime()
session.status ← "Active"
saveToDatabase(session)

// 3. Track session progress
timeElapsed ← 0
while timeElapsed < duration do
    updateProgressBar()
    timeElapsed ← timeElapsed + 1

// 4. Complete session
session.endTime ← currentDateTime()
session.status ← "Completed"

// 5. Award XP
userXP ← fetchUserXP(userId)
userXP ← userXP + 10
updateUserXP(userId, userXP)
display("Session complete! You earned +10 XP")
```

Use Case 2: Set Daily Goal (Daily Goal UI)

When a user sets a daily goal, the system validates input, stores the goal, and tracks progress toward completion.

Algorithm 2: The setDailyGoal() function — triggered when a user sets a daily goal

Input:
userId — the ID of the logged-in user;
goalValue — number of tasks

Output:
A DailyGoal record; error message if invalid

Algorithm: The setDailyGoal() function

```
Input: userId – the ID of the user; goalValue – number of tasks
Output: A DailyGoal record; error message if invalid
goal ← null;
/* 1. validate input                                  */
if goalValue <= 0 then
    display error: "Goal must be greater than 0";
    return;
/* 2. create goal                                     */
goal ← new DailyGoal(userId, goalValue);
goal.progress ← 0;
saveToDatabase(goal);
/* 3. track progress                                  */
completedTasks ← fetchCompletedTasks(userId);
foreach t ∈ completedTasks do
    goal.progress ← goal.progress + 1;
/* 4. check completion                                */
if goal.progress >= goal.target then
    awardXP(userId, 20);
    display: "Daily goal reached!";
/* 5. save updates                                    */
updateGoal(goal);
```

Use Case 3: Block Distracting Websites (Focus Blocking UI)

When a user manages blocked websites, the system updates the list and enforces blocking during focus sessions

Algorithm 3: The manageBlockedWebsites() function — triggered when a user adds or removes a website

Input:
userId — the ID of the logged-in user;
domain — website domain;
action — "ADD" or "REMOVE"

Output:
Updated blocked website list; error message if invalid

Algorithm: The manageBlockedWebsites() function

```
Input: userId – the ID of the user; domain – website domain; action –
       ADD or REMOVE
Output: Updated blocked website list; error message if invalid
blacklist ← null;
/* 1. validate input                                  */
if domain is empty then
    display error: "Domain required";
    return;
/* 2. retrieve blocked websites                       */
blacklist ← fetchBlockedWebsites(userId);
/* 3. update blocked list                             */
if action = ADD then
    if domain not in blacklist then
        add domain to blacklist;
    else
        display: "Website already blocked";
else if action = REMOVE then
    if domain in blacklist then
        remove domain from blacklist;
    else
        display: "Website not found";
/* 4. save updates                                    */
saveToDatabase(blacklist);
/* 5. enforce blocking during active session          */
if focusSession is active then
    foreach site ∈ blacklist do
        blockAccess(site);
/* 6. display confirmation                            */
display: "Blocked website list updated";
```

Use Case 4: Review Focus Session History (Session History UI)

When a user views session history, the system retrieves sessions, calculates summary statistics, and displays results.

Algorithm 4: The loadFocusSessionHistory() function — loads session history for the user

Input:
userId — the ID of the logged-in user

Output:
A list of FocusSession records; summary statistics; message if empty

Algorithm: The loadFocusSessionHistory() function

```
Input: userId – the ID of the user
Output: List of FocusSession records; summary statistics; message if
        empty
sessions ← null;
/* 1. retrieve session data                                  */
sessions ← fetchFocusSessions(userId);
/* 2. check if no sessions exist                             */
if sessions is empty then
    display: "No focus sessions found";
    return;
/* 3. initialize summary values                              */
totalToday ← 0;
totalWeek ← 0;
totalAllTime ← 0;
/* 4. compute statistics                                     */
foreach s ∈ sessions do
    totalAllTime ← totalAllTime + s.duration;
    if s.date >= currentDate() - 1 day then
        totalToday ← totalToday + s.duration;
    if s.date >= currentDate() - 7 days then
        totalWeek ← totalWeek + s.duration;
/* 5. sort sessions                                          */
sort sessions by date descending;
/* 6. display results                                        */
display summary: totalToday, totalWeek, totalAllTime;
display session list;
```

Task Management System

UI MockUp Design for Design 2 3704 Project

Design Notes:

Background: White across all screens

Primary color: Blue for buttons,headers, and highlights

Font sizes: (20 pt) for headers and 12 pt for paragraph text and explanations

Buttons: Primary: colored background with white text; Secondary with gray border

Input Fields: Rounded Corners and consistent width

Font Style: Arial

UI 1: Start Focus Session (UC21 – Focus Session)

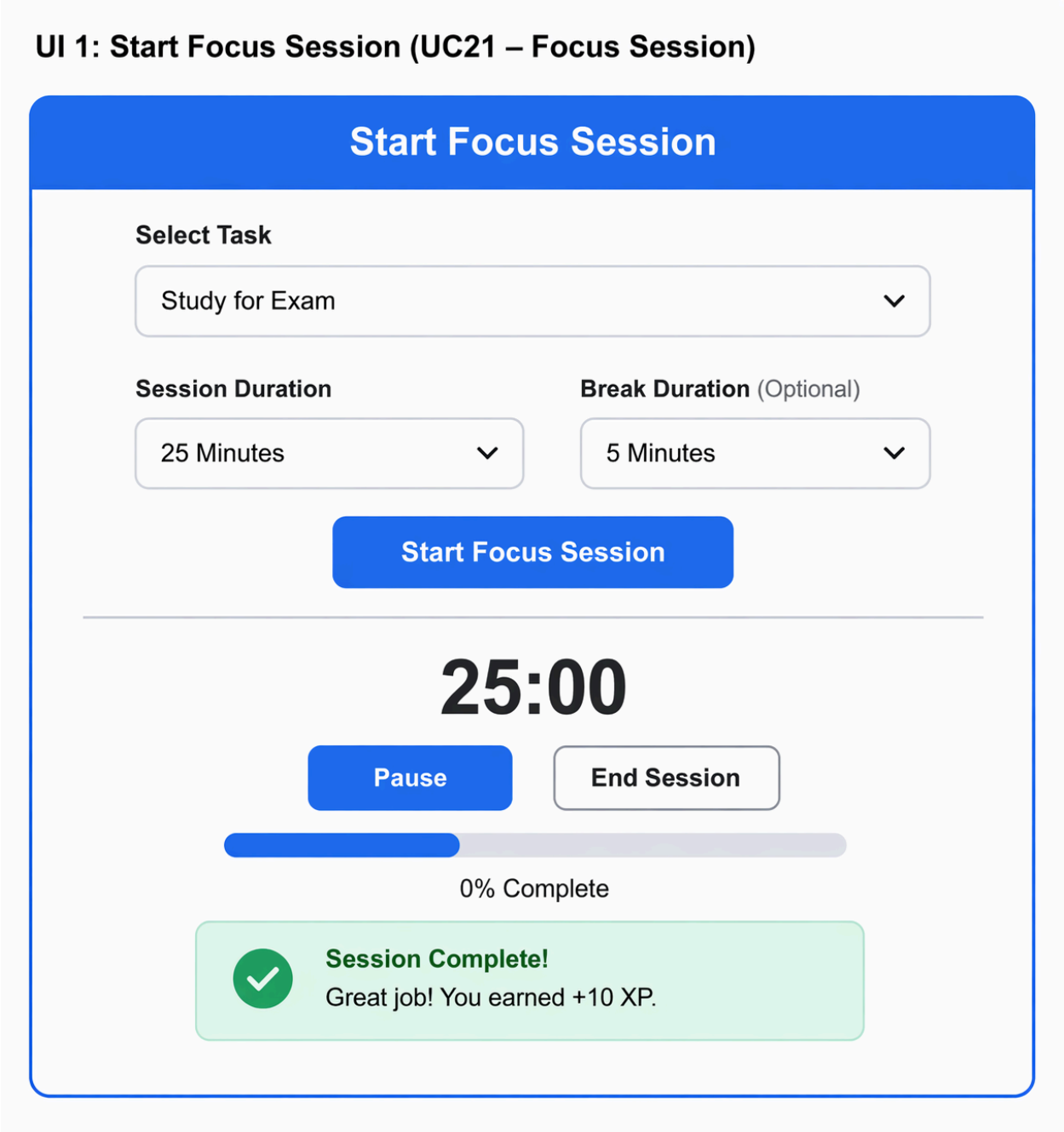
Description:

This screen allows the user to start a focus session by selecting a task and setting a session duration. The system displays a countdown timer and visual progress while the session is active. When the session completes, the user is notified and can choose to take a break or continue working.

Key Fields:

- Task Selection (dropdown or list of user tasks)
- Session Duration (number input or preset options, default 25 minutes)
- Break Duration (optional, default 5 minutes)
- Start Focus Session button
- Pause / End Session controls
- Countdown Timer display
- Progress Bar (visual indicator of time remaining)
- Completion Notification (message when session ends)

Example:



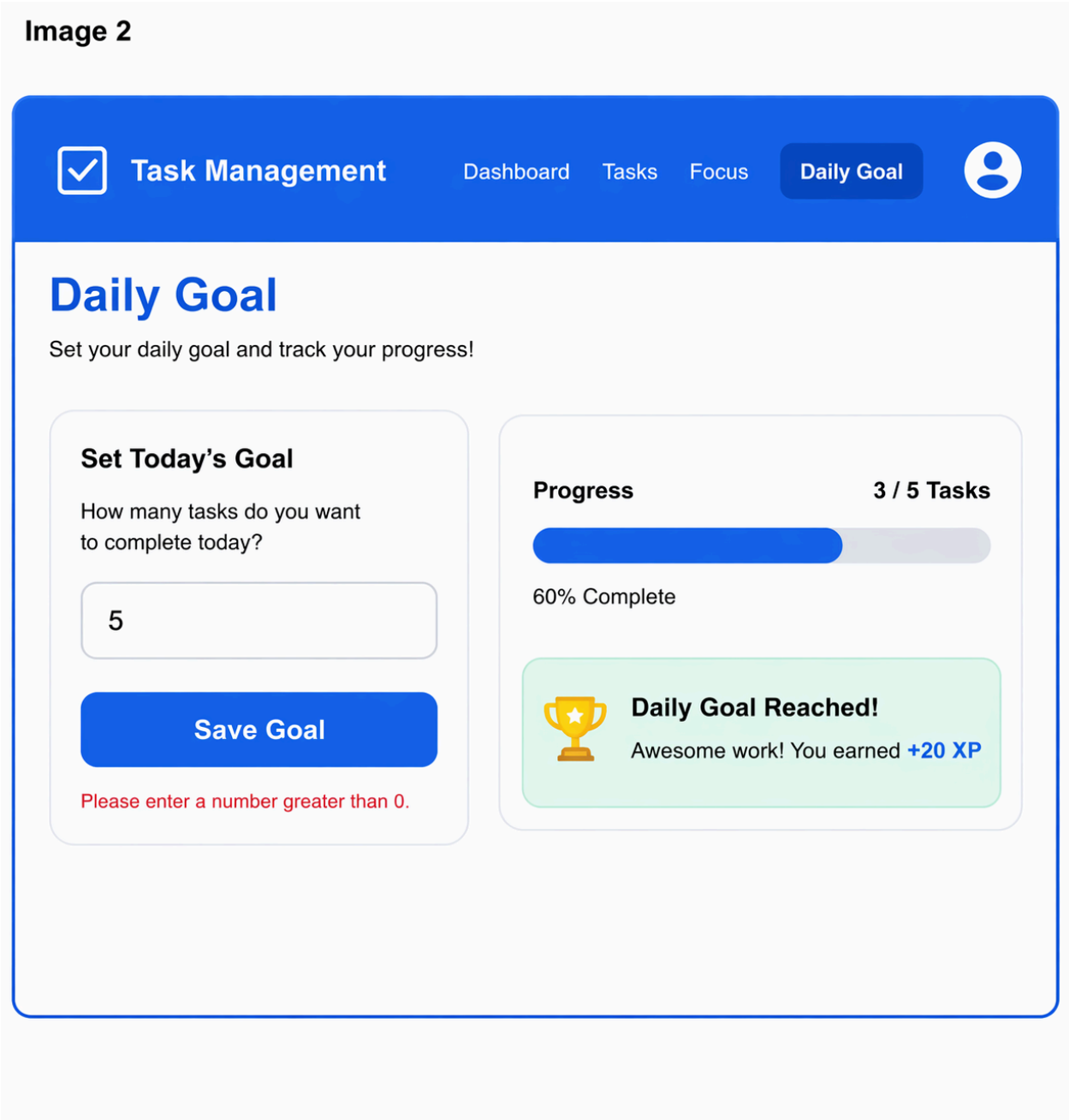
UI 2: Set Daily Goal (UC22 – Daily Goal)

Description:

This screen allows the user to set a daily goal for the number of tasks they want to complete. The system tracks progress toward this goal and updates the progress indicator in real time as tasks are completed. When the goal is reached, the system displays a completion message and awards bonus XP.

Key Elements:

- Daily Goal Input (number field for tasks to complete)
- Save Goal button
- Progress Bar (goal completion)
- Current Progress Display
- Completion Message (displayed when goal is reached)
- Error Message (for invalid input such as 0 or negative values)



UI 3: Block Distracting Websites (UC23 – Focus Blocking)

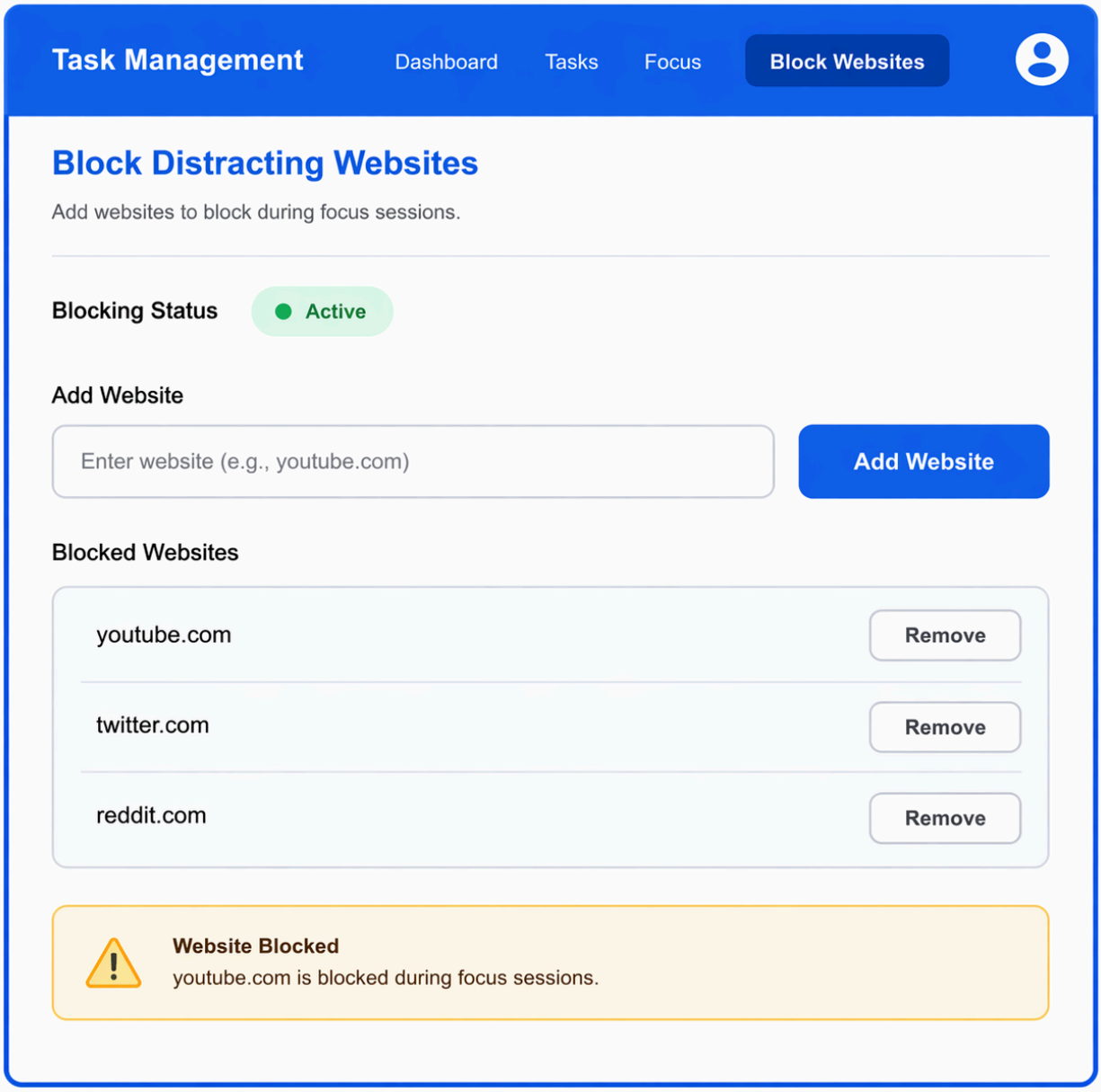
Description:

This screen allows the user to manage a list of distracting websites that will be blocked during focus sessions. The system activates blocking when a session starts and removes restrictions when the session ends to help users stay focused on their tasks.

Key Elements:

- Website Input Field (for entering domain names)
- Add Website button
- Blocked Websites List (displays all added domains)
- Remove Website button (for deleting entries)
- Blocking Status Indicator (shows whether blocking is active)

Image 3



UI 4: Review Focus Session History (UC24 – Session History)

Description:

This screen allows the user to review past focus sessions and track productivity over time. The system displays a list of completed and partial sessions along with summary statistics, helping users analyze their focus habits.

Key Elements:

- Session List (shows date, duration, linked task, and status)
- Status Indicator (Complete or Partial)
- Summary Statistics (total focus time for today, week, and overall)
- Filter Options (by date or task category)
- Empty State Message (displayed if no sessions exist)

Image 4

Task Management Dashboard Tasks Focus **History**

Focus Session History

Review your progress and stay motivated.

Filter: All Tasks | This Week

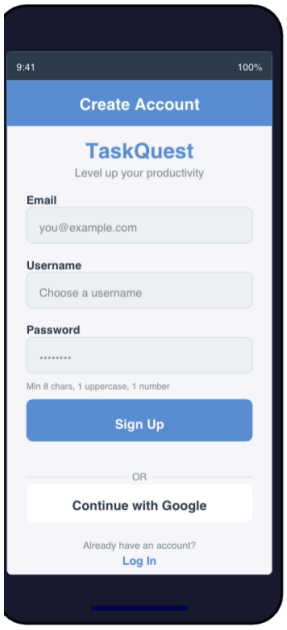
Today	This Week	All Time
1h 30m	**6h 45m**	**24h 15m**

Date	Task	Duration	Status
May 14, 2024 10:00 AM	Finish CS 3704 Project	25 minutes	Complete
May 13, 2024 2:30 PM	Study for Exam	20 minutes	Partial
May 12, 2024 11:00 AM	Write Design Doc	25 minutes	Complete

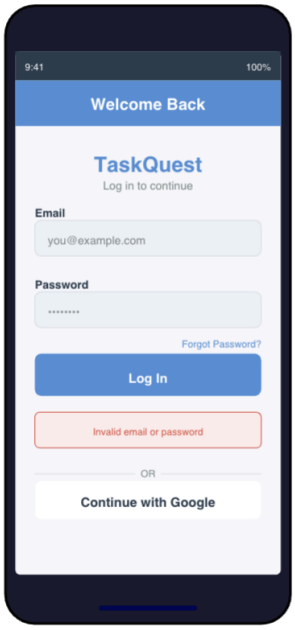
No focus sessions yet.

Start your first focus session to see your history here!

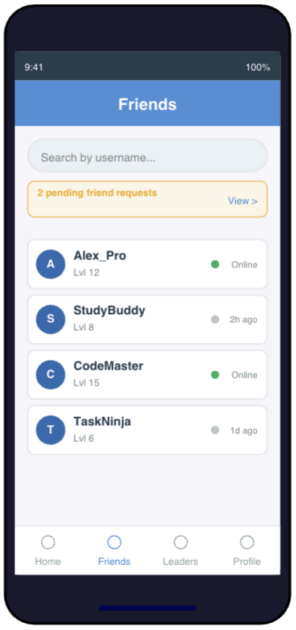
UI Mockups - UC1: Authentication & UC2: Friends List



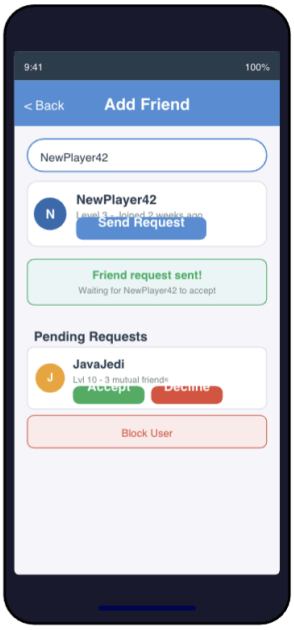
UC1: Sign Up



UC1: Login

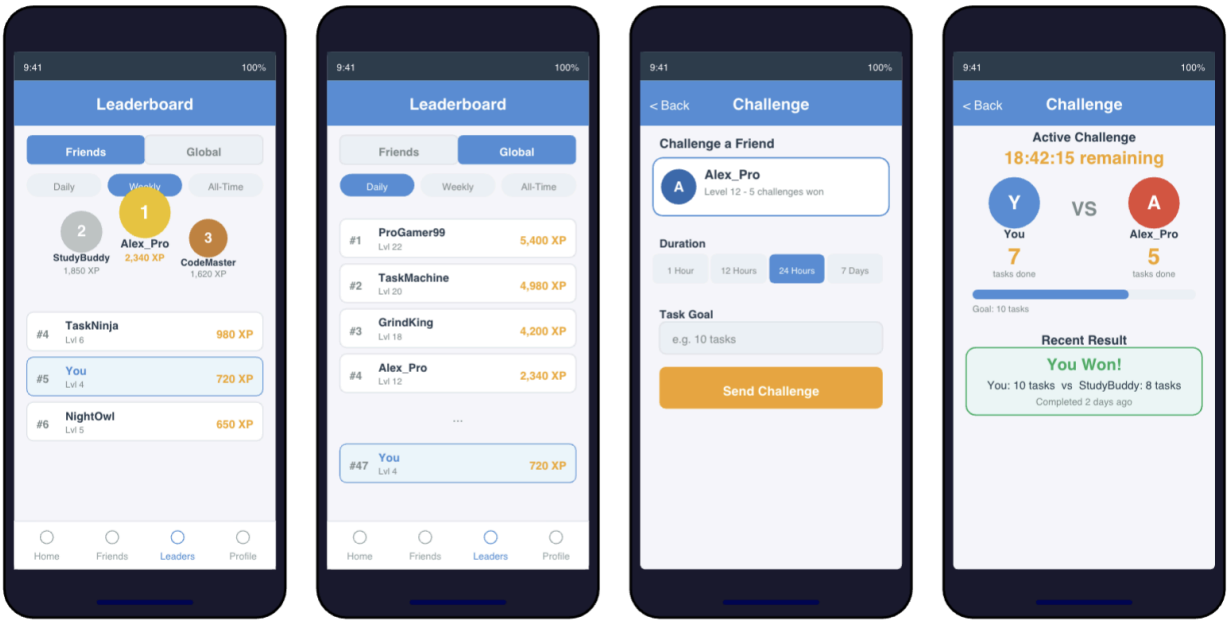


UC2: Friends List



UC2: Add Friend

UI Mockups - UC3: Leaderboard & UC4: 1v1 Challenge



UC3: Friends/Weekly | UC3: Global/Daily | UC4: Create Challenge | UC4: Active / Result

Algorithms - UC1: User Authentication

Algorithm 1: The registerUser() function

Input: email, username, password Output: Success with new User, or error message

```
/* 1. Validate input fields */
if email is empty OR username is empty OR password is empty then
    return error("All fields are required")
if email does not match valid email pattern then
    return error("Invalid email format")
/* 2. Enforce password strength */
if length(password) < 8 OR missing uppercase OR missing digit then
    return error("Password too weak")
/* 3. Check for duplicates */
existingUser ← findUserByEmail(email)
if existingUser is not null then
    return error("Email already registered")
```

```
/* 4. Create account and start session */
hashedPw ← hashPassword(password)
user ← createUser(email, username, hashedPw)
profile ← createDefaultProfile(user)
session ← createSession(user)
return success(user, session)
```

Algorithm 2: The loginUser() function

```
Input: email, password Output: Success with AuthSession, or error message
if email is empty OR password is empty then
   return error("Email and password are required")
user ← findUserByEmail(email)
if user is null OR verifyPassword(password, user.passwordHash) is False then
   return error("Invalid email or password")
session ← createSession(user)
logAuthEvent(user, "LOGIN_SUCCESS")
return success(user, session)
```

Algorithms - UC2: Friends List & UC3: Leaderboard

Algorithm 3: The sendFriendRequest() function

```
Input: senderId, recipientUsername Output: Success with FriendRequest, or error
if recipientUsername is empty then
   return error("Username is required")
recipient ← findUserByUsername(recipientUsername)
if recipient is null then return error("No user found")
if recipient.id = senderId then return error("Cannot add yourself")
if areFriends(senderId, recipient.id) then return error("Already friends")
if isBlocked(senderId, recipient.id) then return error("Cannot send request")
if findPendingRequest(senderId, recipient.id) is not null then
   return error("Request already sent")
request ← createFriendRequest(senderId, recipient.id, PENDING)
sendNotification(recipient, "New friend request")
return success(request)
```

Algorithm 4: The respondToFriendRequest() function

```
Input: requestId, userId, action (ACCEPT or DECLINE) Output: Friendship or error message
request ← findRequestById(requestId)
if request is null OR request.receiverId ≠ userId then
```

```
    return error("Invalid request")
if daysSince(request.createdAt) > 7 then
    request.status ← EXPIRED; return error("Request expired")
if action = ACCEPT then
    friendship ← createFriendship(request.senderId, request.receiverId)
    request.status ← ACCEPTED; return success(friendship)
else request.status ← DECLINED; return success("Request declined")
```

Algorithm 5: The getLeaderboard() function

```
Input: userId, scope (FRIENDS_ONLY or GLOBAL), period (DAILY, WEEKLY, ALL_TIME)
Output: Ranked list of leaderboard entries
/* 1. Determine users in scope */
if scope = FRIENDS_ONLY then userIds ← getFriendIds(userId) ∪ {userId}
else userIds ← getAllActiveUserIds()
/* 2. Exclude private profiles */
foreach uid in userIds do
    if getProfile(uid).isPrivate AND uid ≠ userId then remove uid
/* 3. Compute XP for each user */
entries ← empty list; startDate ← computePeriodStart(period)
foreach uid in userIds do
    xp ← sumXpEarnedSince(uid, startDate)
    tasks ← countTasksCompletedSince(uid, startDate)
    add {userId: uid, xp: xp, tasks: tasks} to entries
/* 4. Sort, rank, and return */
sort entries by xp in descending order
foreach i from 1 to length(entries) do entries[i].rank ← i
if entries is empty then return message("No activity yet")
return entries with userId highlighted
```

Algorithms - UC4: 1v1 Task Challenge

Algorithm 6: The issueChallenge() function

```
Input: challengerId, opponentId, durationHours, taskGoal Output: Success with Challenge, or
error message
/* 1. Verify participants are friends */
if NOT areFriends(challengerId, opponentId) then
    return error("You can only challenge friends")
/* 2. Validate challenge parameters */
if durationHours ≤ 0 OR taskGoal ≤ 0 then
    return error("Duration and goal must be positive")
```

```
if taskGoal > 1000 then
   return error("Task goal exceeds maximum")
/* 3. Create and send challenge */
challenge ← createChallenge(challengerId, opponentId, durationHours, taskGoal)
challenge.status ← PENDING
sendNotification(opponent, "New challenge received")
return success(challenge)
```

Algorithm 7: The resolveChallenge() function

Input: challengeId — challenge whose duration has ended Output: ChallengeResult with winner, or tie declaration

```
challenge ← findChallengeById(challengeId)
if challenge.status ≠ ACTIVE then
   return error("Challenge is not active")
if currentTime() < challenge.endTime then
   return error("Challenge still in progress")
/* Count completed tasks for both users */
cTasks ← countTasksInPeriod(challenge.challengerId, startTime, endTime)
oTasks ← countTasksInPeriod(challenge.opponentId, startTime, endTime)
/* Determine winner */
if cTasks > oTasks then
   winner ← challenge.challengerId; isTie ← False
else if oTasks > cTasks then
   winner ← challenge.opponentId; isTie ← False
else
   winner ← null; isTie ← True
/* Finalize and notify both users */
result ← createChallengeResult(challenge, winner, cTasks, oTasks, isTie)
challenge.status ← COMPLETED
notifyBothUsers(challenge, result)
return result
```

Avatar Creations and customization

The avatar creation algorithm validates all user-supplied parameters (name length, valid color code, valid outfit selection) before creating or updating the stored avatar record. Each customization parameter is applied via a uniform setter, allowing new fields to be added without changing the control flow.

Algorithm 1: createAvatar() — Avatar Creation and Customization

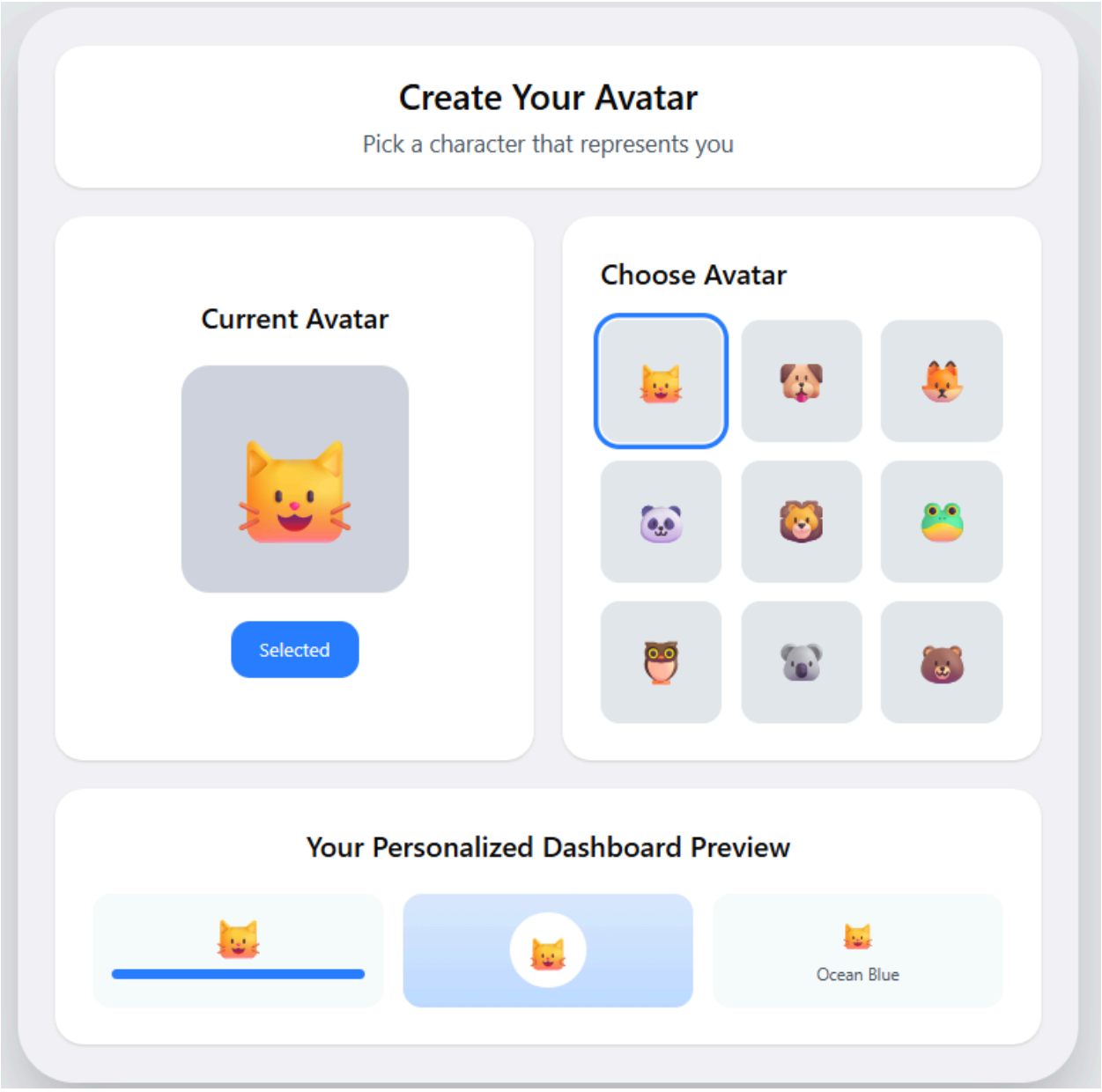
```
Input: userId, avatarParams /* name, bodyType, skinColor, hairStyle,
outfit */
Output: avatar /* newly created or updated avatar object */

1.1  if avatarParams.name is empty or length > 30 then
1.2    | return error("Name must be 1-30 characters")
1.3  if avatarParams.skinColor not in VALID_COLORS then
1.4    | return error("Invalid skin color selection")
1.5  if avatarParams.outfit not in VALID_OUTFITS then
1.6    | return error("Invalid outfit selection")

/* Check if user already has an avatar */
1.7  existingAvatar <- fetchAvatar(userId)
1.8  if existingAvatar != null then
1.9    | avatar <- existingAvatar
1.10 else
1.11   | avatar <- newAvatar(userId)

/* Apply each customization parameter */
1.12 foreach param in avatarParams do
1.13   | avatar.set(param.key, param.value)

/* Persist and return */
1.14 saveAvatar(userId, avatar)
1.15 return avatar
```



Theme System

The theme system validates that the requested theme exists and that the user's subscription plan permits access. Design tokens (colors, fonts, spacing) are iterated and written to the user's CSS variable map, then broadcast to any open client sessions so the UI updates in real time

Algorithm 2: applyTheme() — Theme System

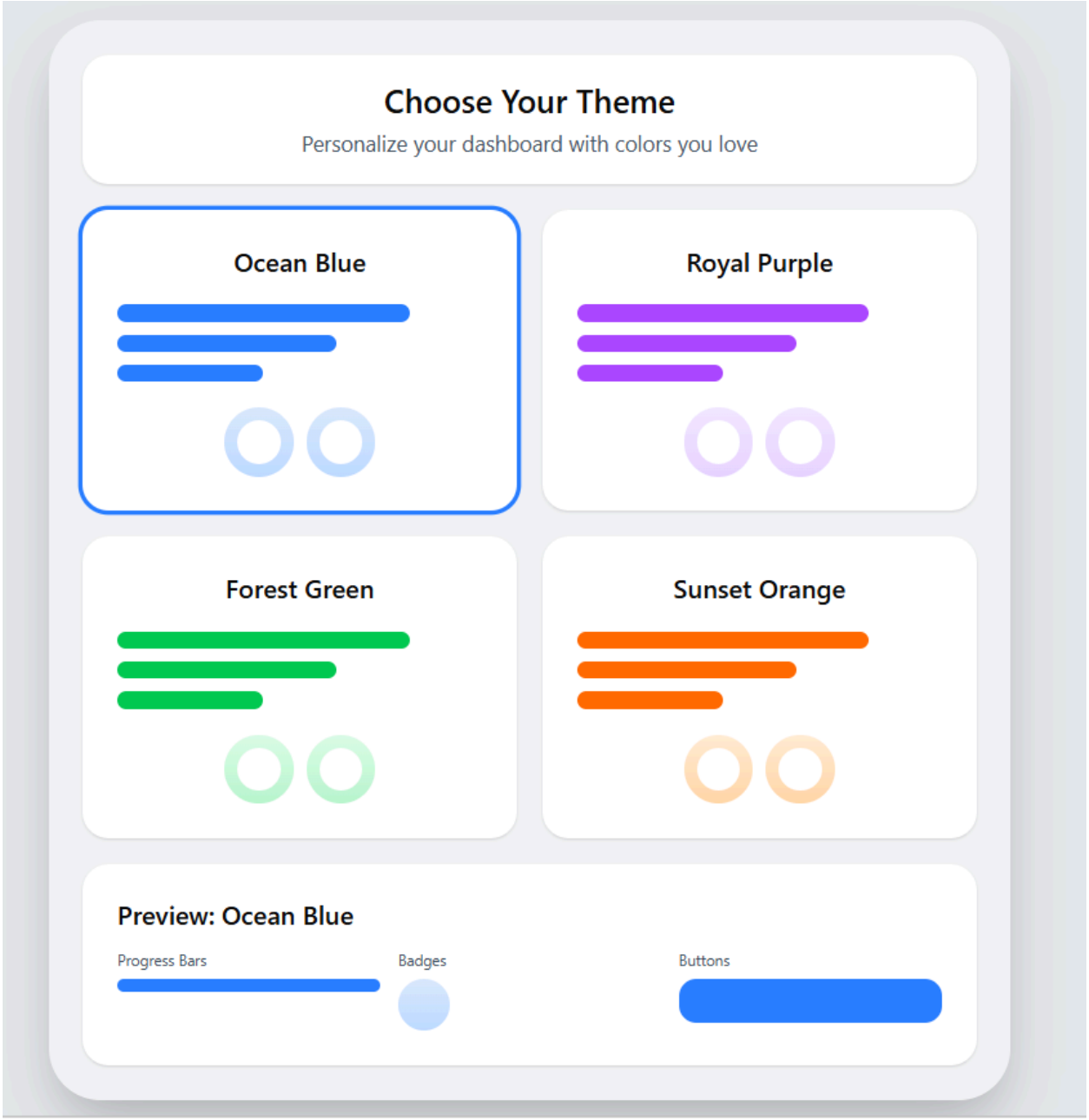
```
Input: userId, themeId /* selected theme identifier */
Output: updatedProfile /* user profile with new theme applied */

/* Validate theme exists and is accessible */
```

```
2.1  theme <- lookupTheme(themeId)
2.2  if theme = null then
2.3    | return error("Theme not found")
2.4  userPlan <- getUserPlan(userId)
2.5  if theme.isPremium = True and userPlan != "premium" then
2.6    | return error("Upgrade required to use premium themes")

/* Apply color palette and font tokens */
2.7  profile <- getProfile(userId)
2.8  profile.theme <- themeId
2.9  foreach token in theme.designTokens do
2.10   | profile.cssVars.set(token.name, token.value)

/* Persist and broadcast change to open sessions */
2.11 saveProfile(userId, profile)
2.12 broadcastThemeChange(userId, theme.designTokens)
2.13 return profile
```



Customizable dashboard

The dashboard layout algorithm enforces a maximum widget count, validates that every widget's position and size are within legal bounds, and detects overlapping placements before persisting the layout. Widgets that the user's plan does not include are rejected with an informative error.

Algorithm 3: saveDashboardLayout() — Customizable Dashboard

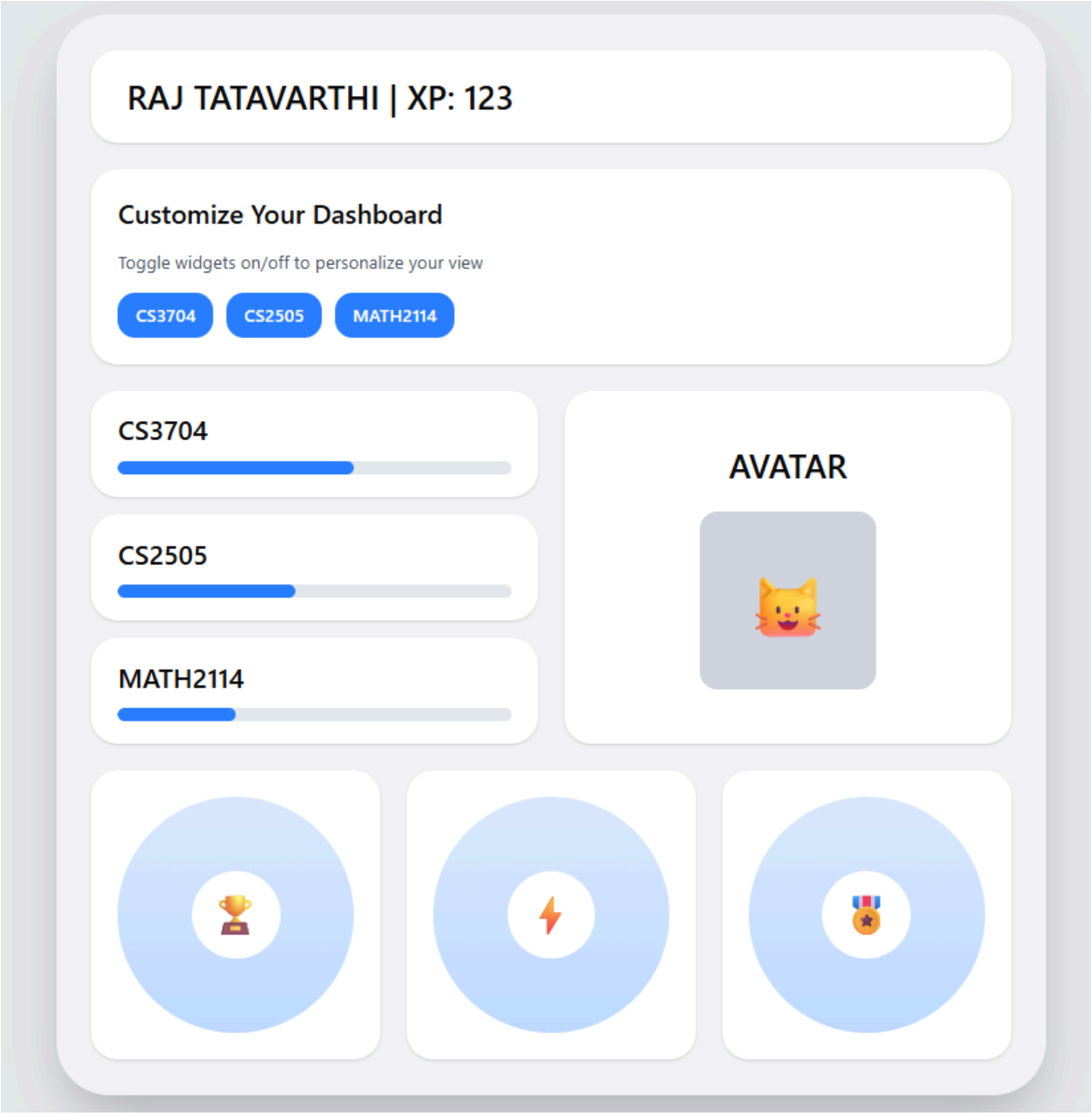
```
Input: userId, widgetList /* array of {widgetId, x, y, w, h} */
Output: layout /* validated and saved dashboard layout */

/* Sanity check: widget count limit */
3.1  if |widgetList| > MAX_WIDGETS then
3.2    | return error("Exceeds max widget limit of " + MAX_WIDGETS)

/* Validate each widget's bounds and permissions */
3.3  foreach widget in widgetList do
3.4    | if widget.x < 0 or widget.y < 0 then
3.5    |   | return error("Widget position out of bounds: " +
widget.widgetId)
3.6    | if widget.w < 1 or widget.h < 1 then
3.7    |   | return error("Widget size invalid: " + widget.widgetId)
3.8    | if not isWidgetAllowed(userId, widget.widgetId) then
3.9    |   | return error("Widget not available for this user")

/* Check for overlapping widgets */
3.10 occupiedCells <- empty set
3.11 foreach widget in widgetList do
3.12   | cells <- computeCells(widget.x, widget.y, widget.w, widget.h)
3.13   | if cells intersects occupiedCells then
3.14   |   | return error("Widget overlap detected")
3.15   | occupiedCells <- occupiedCells UNION cells

3.16 layout <- buildLayout(widgetList)
3.17 saveLayout(userId, layout)
3.18 return layout
```



Onboarding Tutorial

The onboarding tutorial algorithm loads or initializes progress state, verifies that prerequisite steps are satisfied before executing the current step, records completion, and returns the next step identifier. When all steps are complete the algorithm marks onboarding as finished and returns a COMPLETE sentinel.

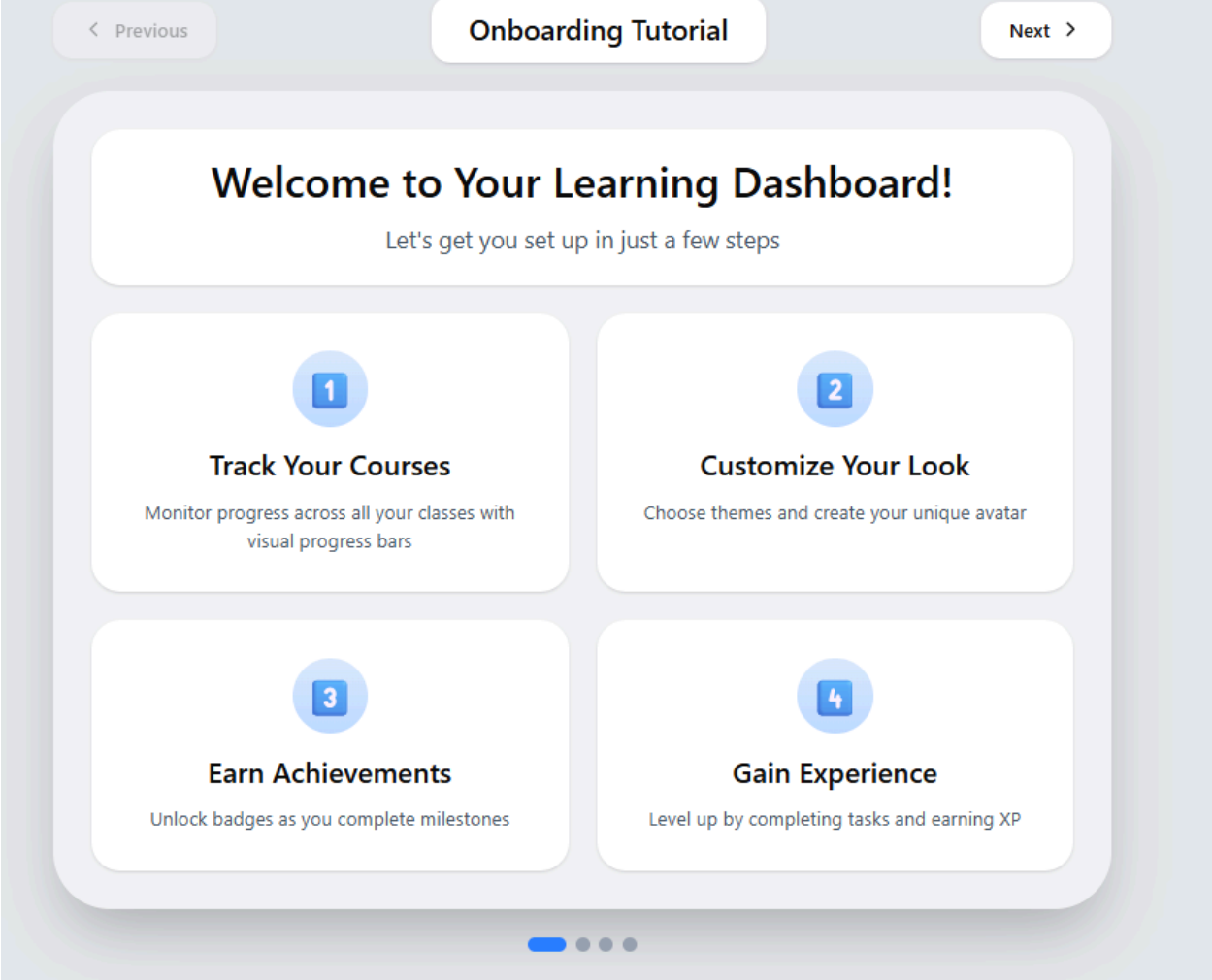
Algorithm 4: runOnboardingTutorial() — Onboarding Tutorial

```
Input: userId, stepId /* current tutorial step to process */
Output: nextStep /* next tutorial step, or COMPLETE if finished */

/* Load user's onboarding progress */
4.1  progress <- getOnboardingProgress(userId)
4.2  if progress = null then
4.3    | progress <- initializeProgress(userId)
```

```
/* Validate step */
4.4  step <- lookupStep(stepId)
4.5  if step = null then
4.6    | return error("Invalid tutorial step")
4.7  if step.prerequisite not in progress.completedSteps then
4.8    | return error("Prerequisite step not completed")

/* Execute step action and record completion */
4.9  result <- executeStep(step, userId)
4.10 if result.success = True then
4.11   | progress.completedSteps <- progress.completedSteps UNION {stepId}
4.12   | if |progress.completedSteps| = |ALL_STEPS| then
4.13   |   | markOnboardingComplete(userId)
4.14   |   | return COMPLETE
4.15   | nextStep <- getNextStep(stepId)
4.16   | saveProgress(userId, progress)
4.17   | return nextStep
4.18 else
4.19   | return error("Step action failed: " + result.message)
```


Previous
Onboarding Tutorial
Next
Welcome to Your Learning Dashboard!
Let's get you set up in just a few steps
1
Track Your Courses
Monitor progress across all your classes with visual progress bars
2
Customize Your Look
Choose themes and create your unique avatar
3
Earn Achievements
Unlock badges as you complete milestones
4
Gain Experience
Level up by completing tasks and earning XP

Task Management System

UI Design Mockups
Core Task System - CS 2104 Software Engineering Project

Design Consistency

Element	Standard
Background	White (#FFFFFF) across all screens
Primary Color	Blue (#5B9BD5) for headers, buttons, accents
Font Sizes	4 sizes only: 20px (title), 16px (heading), 14px (body), 12px (small/labels)
Input Fields	Rounded corners (6px), light gray border
Buttons	Primary (blue fill, white text), Secondary (white fill, gray border)

UI 1: Create / Edit Task (UC1 - Manage Task)

Description: The primary task creation and editing interface. Users can input task title (required), description, select due date via date picker, and choose priority level (High/Medium/Low) from a dropdown. The form also provides quick access to category assignment, tag selection, recurring task toggle, and subtask management. Save and Cancel buttons are prominently displayed at the bottom.

Key Input Fields:

- Task Title (required, text input)
- Description (multiline text area)
- Due Date (date picker)
- Priority (dropdown: High/Medium/Low)
- Category (dropdown selection)
- Tags (multi-select)
- Recurring toggle (checkbox)
- Subtasks area (expandable list)

Create / Edit Task

Task Title *

Enter task title...

Description

Enter description...

Due Date

MM/DD/YYYY

Priority

Select priority...

Category

Tags

Make Recurring

+ Add Subtask

Save Task

Cancel

UI 2: Organize Tasks (UC2 - Organize Task)

Description: A management interface for organizational elements. Displays existing categories with color indicators and edit controls. Shows tags as colored chips for quick visual identification. Includes an "Add Category" and "Add Tag" button for creating new organizational elements. The subtasks section shows a checklist view with completion toggles and add functionality.

Key Features:

- Category list with color swatches and edit icons
- Tag chips with color coding (Urgent, Review, Meeting, Bug)
- Add Category / Add Tag buttons
- Subtask checklist with toggle completion

Organize Tasks

Categories

- Work
- Personal
- Shopping

Tags

+ Add Category

+ Add Tag

Subtasks

Task 1 subtask + Add

UI 3: Recurring Schedule (UC3 - Configure Recurring Task)

Description: A dedicated configuration screen for setting up recurring tasks. Users select frequency type (Daily/Weekly/Custom) via toggle buttons. For weekly recurrence, a day-of-week selector (S M T W T F S) allows choosing specific days. End conditions can be set via end date picker or occurrence count. A live preview panel shows the next scheduled occurrences.

Key Input Fields:

- Frequency selector (Daily/Weekly/Custom toggle)
- Interval input (every X days/weeks)
- Day-of-week selector (circular buttons for each day)
- End Date (optional date picker)
- Repeat Count (optional number input)
- Schedule Preview (read-only, shows next 4+ occurrences)

Recurring Schedule

Frequency

Daily | **Weekly** | Custom

Repeat Interval

Every 1 week(s)

Repeat On Days

S M T W TR F SR

End Date

Select date...

Occurrences

10 times

Schedule Preview

Next occurrences:

Mon, Apr 14, 2026
Wed, Apr 16, 2026
Fri, Apr 18, 2026
Mon, Apr 21, 2026

+ 6 more

Save Schedule

UI 4: Search & Filter (UC4 - Search and Filter)

Description: A powerful search and filtering interface for finding tasks. Features a prominent search bar for keyword entry. Filter section provides dropdowns for Priority, Category, Tag, and Date Range filters. Sort controls allow ordering by Due Date, Priority, or Created Date. Results display in a scrollable list with priority indicators (color-coded dots) and due date information.

Key Features:

- Search bar with placeholder text
- Filter dropdowns (Priority, Category, Tag, Date Range)
- Sort options (Due Date | Priority | Created)
- Results list showing task title, due date, and priority badge
- Result count indicator (e.g., "4 tasks found")

Search & Filter

Search tasks...

Filters

Priority

Category

Tag

Date Range

Sort by: Due Date | Priority | Created

4 tasks found

Complete project report
Due: Today - HIGH

Review pull requests
Due: Tomorrow - MEDIUM

Team meeting prep
Due: Apr 16 - LOW

Update documentation
Due: Apr 18 - LOW

Task Management System
Algorithm Design Document

CS 2104 - Software Engineering

Contents

1 UC1: Manage Task

This use case allows users to create, edit, and delete tasks with attributes including title, description, due date, and priority. The system validates all inputs and handles associated data (subtasks, tags, recurring schedules) appropriately.

1.1 Algorithm 1.1: createTask()

Description: This algorithm handles the creation of a new task. It validates all required fields (title is mandatory), checks input constraints (title length, due date not in past), assigns default values where needed, and associates the task with optional categories and tags before persisting to the database.

Algorithm 1: The **createTask()** function

```
Input: taskData—the task form data containing title, description, dueDate, priority, categoryId, tags
Output: The created Task object or error message
// 1. Validate required fields
if taskData.title is empty or null then
    return error("Title is required")
if length(taskData.title) > 255 then
    return error("Title must be 255 characters or less")
// 2. Validate due date if provided
if taskData.dueDate is not null then
    if taskData.dueDate < currentDate() then
        return error("Due date cannot be in the past")
// 3. Validate and set priority
if taskData.priority ∉ {HIGH, MEDIUM, LOW} then
    taskData.priority ← MEDIUM ;                                   // default priority
// 4. Create the task object
task ← new Task()
task.id ← generateUniqueId()
task.title ← trim(taskData.title)
task.description ← taskData.description
task.dueDate ← taskData.dueDate
task.priority ← taskData.priority
task.isComplete ← False
task.createdAt ← currentTimestamp()
task.updatedAt ← currentTimestamp()
// 5. Associate category if provided
if taskData.categoryId is not null then
    category ← findCategoryById(taskData.categoryId)
    if category is null then
        return error("Category not found")
    task.category ← category
// 6. Associate tags if provided
foreach tagId in taskData.tags do
    tag ← findTagById(tagId)
    if tag is not null then
        addTagToTask(task, tag)
// 7. Save and return
saveToDatabase(task)
return task
```

1.2 Algorithm 1.2: deleteTask()

Description: This algorithm handles task deletion. It first verifies the task exists, then checks if it has a recurring schedule (offering options to delete single or all occurrences). It properly cascades deletion to associated subtasks and removes tag associations before deleting the task itself.

Algorithm 2: The **deleteTask()** function

```
Input: taskId—the ID of the task to delete, deleteAll—boolean flag for recurring tasks
Output: Success confirmation or error message
// 1. Validate task exists
task ← findTaskById(taskId)
if task is null then
    return error("Task not found")
// 2. Check if task has recurring schedule
if task.recurringSchedule is not null then
    if deleteAll is True then
        // Delete all future occurrences
        occurrences ← findAllOccurrences(task.recurringSchedule.id)
        foreach occurrence in occurrences do
            deleteSubtasks(occurrence.id)
            removeFromDatabase(occurrence)
        removeFromDatabase(task.recurringSchedule)
    else
        // Delete only this occurrence
        deleteSubtasks(task.id)
        removeFromDatabase(task)
        return success("Single occurrence deleted")
// 3. Delete associated subtasks
subtasks ← findSubtasksByTaskId(taskId)
foreach subtask in subtasks do
    removeFromDatabase(subtask)
// 4. Remove tag associations
removeTaskTagAssociations(taskId)
// 5. Delete the task
removeFromDatabase(task)
return success("Task deleted successfully")
```

2 UC2: Organize Task

This use case allows users to organize their tasks using categories (with custom colors), tags (many-to-many relationship), and subtasks (ordered checklist items). Users can create organizational elements and associate them with tasks.

2.1 Algorithm 2.1: createCategory()

Description: This algorithm creates a new category for organizing tasks. It validates the category name (required, max 100 characters), validates the hex color format, checks for duplicate names within the user's categories, and creates the category with a default color if none is specified.

Algorithm 3: The **createCategory()** function

```
Input: userId—the user's ID, name—category name, color—hex color code
Output: The created Category object or error message
// 1. Validate inputs
if name is empty or null then
    return error("Category name is required")
if length(name) > 100 then
    return error("Category name must be 100 characters or less")
// 2. Validate color format
if color is not null then
    if not isValidHexColor(color) then
        return error("Invalid color format. Use hex: #RRGGBB")
else
    color ← "#5B9BD5" ;                                   // default color
// 3. Check for duplicate category name
existing ← findCategoryByName(userId, name)
if existing is not null then
    return error("Category with this name already exists")
// 4. Create category object
category ← new Category()
category.id ← generateUniqueId()
category.userId ← userId
category.name ← trim(name)
category.color ← color
// 5. Save and return
saveToDatabase(category)
return category
```

2.2 Algorithm 2.2: addSubtask()

Description: This algorithm adds a subtask to an existing task. It validates that the parent task exists, validates the subtask title, calculates the appropriate sort order by finding the maximum existing sort order and incrementing it, then creates and saves the subtask.

Algorithm 4: The **addSubtask()** function

```
Input: taskId—parent task ID, title—subtask title
Output: The created Subtask object or error message
// 1. Validate parent task exists
parentTask ← findTaskById(taskId)
if parentTask is null then
    return error("Parent task not found")
// 2. Validate subtask title
if title is empty or null then
    return error("Subtask title is required")
if length(title) > 255 then
    return error("Subtask title must be 255 characters or less")
// 3. Calculate sort order
existingSubtasks ← findSubtasksByTaskId(taskId)
maxSortOrder ← 0
foreach subtask in existingSubtasks do
    if subtask.sortOrder > maxSortOrder then
        maxSortOrder ← subtask.sortOrder
// 4. Create subtask object
subtask ← new Subtask()
subtask.id ← generateUniqueId()
subtask.taskId ← taskId
subtask.title ← trim(title)
subtask.isComplete ← False
subtask.sortOrder ← maxSortOrder + 1
// 5. Save and return
saveToDatabase(subtask)
return subtask
```

3 UC3: Configure Recurring Task

This use case allows users to set up recurring schedules for tasks. Users can choose daily, weekly, or custom frequency, specify intervals, select days of the week for weekly recurrence, and set end conditions (end date or number of occurrences). The system automatically generates new task occurrences when recurring tasks are completed.

3.1 Algorithm 3.1: configureRecurringSchedule()

Description: This algorithm configures a recurring schedule for a task. It validates the frequency type, interval (must be at least 1), days of week for weekly frequency, and end conditions (either end date OR repeat count, not both). It creates the schedule and links it to the task.

Algorithm 5: The **configureRecurringSchedule()** function

```
Input: taskId—the task ID, scheduleData—recurring schedule configuration
Output: The RecurringSchedule object or error message
// 1. Validate task exists
task ← findTaskById(taskId)
if task is null then
    return error("Task not found")
// 2. Validate frequency
if scheduleData.frequency ∉ {DAILY, WEEKLY, CUSTOM} then
    return error("Invalid frequency. Must be DAILY, WEEKLY, or CUSTOM")
// 3. Validate interval
if scheduleData.interval < 1 then
    return error("Interval must be at least 1")
// 4. Validate days of week for WEEKLY frequency
if scheduleData.frequency = WEEKLY then
    if scheduleData.daysOfWeek is empty then
        return error("Select at least one day for weekly recurrence")
    foreach day in scheduleData.daysOfWeek do
        if day ∉ {MON, TUE, WED, THU, FRI, SAT, SUN} then
            return error("Invalid day of week: " + day)
// 5. Validate end conditions (mutually exclusive)
if scheduleData.endDate is not null AND scheduleData.repeatCount is not null then
    return error("Specify either end date OR repeat count, not both")
// 6. Create recurring schedule
schedule ← new RecurringSchedule()
schedule.id ← generateUniqueId()
schedule.frequency ← scheduleData.frequency
schedule.interval ← scheduleData.interval
schedule.endDate ← scheduleData.endDate
schedule.repeatCount ← scheduleData.repeatCount
schedule.daysOfWeek ← join(scheduleData.daysOfWeek, ",")
// 7. Link to task and save
saveToDatabase(schedule)
task.scheduleId ← schedule.id
updateDatabase(task)
return schedule
```

3.2 Algorithm 3.2: generateNextOccurrence()

Description: This algorithm generates the next occurrence of a recurring task when the current occurrence is marked complete. It checks if the schedule has expired (by end date or repeat count), calculates the next due date based on frequency type, creates a new task occurrence with copied properties, and duplicates subtasks.

Algorithm 6: The **generateNextOccurrence()** function

```
Input: task—the completed recurring task
Output: The new Task occurrence or null if schedule expired
// 1. Get recurring schedule
schedule ← findScheduleById(task.scheduleId)
if schedule is null then
    return null
// 2. Check if schedule has expired
if schedule.endDate is not null AND currentDate() > schedule.endDate then
    return null ;                                          // schedule expired by date
if schedule.repeatCount is not null then
    completedCount ← countCompletedOccurrences(schedule.id)
    if completedCount ≥ schedule.repeatCount then
        return null ;                                      // max occurrences reached
// 3. Calculate next due date based on frequency
nextDueDate ← task.dueDate
if schedule.frequency = DAILY then
    nextDueDate ← addDays(nextDueDate, schedule.interval)
else
    if schedule.frequency = WEEKLY then
        daysOfWeek ← split(schedule.daysOfWeek, ",")
        nextDueDate ← findNextWeekday(nextDueDate, daysOfWeek, schedule.interval)
    else
        // CUSTOM frequency
        nextDueDate ← addDays(nextDueDate, schedule.interval)
// 4. Create new task occurrence
newTask ← new Task()
newTask.id ← generateUniqueId()
newTask.title ← task.title
newTask.description ← task.description
newTask.priority ← task.priority
newTask.categoryId ← task.categoryId
newTask.scheduleId ← task.scheduleId
newTask.dueDate ← nextDueDate
newTask.isComplete ← False
// 5. Copy subtasks to new occurrence
foreach subtask in findSubtasksByTaskId(task.id) do
    newSubtask ← copySubtask(subtask, newTask.id)
    saveToDatabase(newSubtask)
saveToDatabase(newTask)
return newTask
```

4 UC4: Search and Filter

This use case allows users to find tasks using keyword search (on title and description), filter by multiple criteria (priority, category, tags, due date range), and sort results by various attributes. The system supports combining multiple filters for precise task retrieval.

4.1 Algorithm 4.1: searchTasks()

Description: This algorithm performs a comprehensive search across tasks. It builds a database query by applying keyword search on title/description fields, then applies optional filters for priority, category, tags, and due date range. Finally, it applies sorting and executes the query.

Algorithm 7: The **searchTasks()** function

```
Input: userId—the user's ID, searchCriteria—search parameters
Output: List of matching Task objects
// 1. Initialize query
query ← buildBaseQuery(userId)
// 2. Apply keyword search on title and description
if searchCriteria.keyword is not null AND length(searchCriteria.keyword) > 0 then
    keyword ← toLowerCase(trim(searchCriteria.keyword))
    query ← addKeywordFilter(query, keyword, ["title", "description"])
// 3. Apply priority filter
if searchCriteria.priority is not null then
    if searchCriteria.priority ∈ {HIGH, MEDIUM, LOW} then
        query ← addFilter(query, "priority", searchCriteria.priority)
// 4. Apply category filter
if searchCriteria.categoryId is not null then
    query ← addFilter(query, "categoryId", searchCriteria.categoryId)
// 5. Apply tag filter
if searchCriteria.tagIds is not null AND length(searchCriteria.tagIds) > 0 then
    query ← addTagFilter(query, searchCriteria.tagIds)
// 6. Apply due date range filter
if searchCriteria.dueDateStart is not null then
    query ← addFilter(query, "dueDate ≥", searchCriteria.dueDateStart)
if searchCriteria.dueDateEnd is not null then
    query ← addFilter(query, "dueDate ≤", searchCriteria.dueDateEnd)
// 7. Apply sorting
if searchCriteria.sortBy ∈ {dueDate, priority, createdAt} then
    sortOrder ← searchCriteria.sortOrder = "DESC" ? DESC : ASC
    query ← addSort(query, searchCriteria.sortBy, sortOrder)
else
    query ← addSort(query, "dueDate", ASC) ;                    // default
// 8. Execute query and return results
results ← executeQuery(query)
return results
```

4.2 Algorithm 4.2: filterTasks()

Description: This algorithm provides in-memory filtering of a task list. It iterates through each task and applies filter criteria sequentially (priority, category, completion status, due date range, tags). Tasks that pass all applicable filters are included in the result set.

Algorithm 8: The **filterTasks()** function

```
Input: tasks—list of Task objects, filterCriteria—filter parameters
Output: Filtered list of Task objects
filteredTasks ← empty list
foreach task in tasks do
    includeTask ← True
    // Check priority filter
    if filterCriteria.priority is not null then
        if task.priority ≠ filterCriteria.priority then
            includeTask ← False
    // Check category filter
    if includeTask AND filterCriteria.categoryId is not null then
        if task.categoryId ≠ filterCriteria.categoryId then
            includeTask ← False
    // Check completion status filter
    if includeTask AND filterCriteria.isComplete is not null then
        if task.isComplete ≠ filterCriteria.isComplete then
            includeTask ← False
    // Check due date range
    if includeTask AND filterCriteria.dueDateStart is not null then
        if task.dueDate < filterCriteria.dueDateStart then
            includeTask ← False
    if includeTask AND filterCriteria.dueDateEnd is not null then
        if task.dueDate > filterCriteria.dueDateEnd then
            includeTask ← False
    // Check tag filter
    if includeTask AND filterCriteria.tagIds is not null then
        hasMatchingTag ← False
        foreach tagId in filterCriteria.tagIds do
            if taskHasTag(task, tagId) then
                hasMatchingTag ← True
                break
        if not hasMatchingTag then
            includeTask ← False
    // Add to results if passes all filters
    if includeTask then
        append(filteredTasks, task)
return filteredTasks
```

QuestLog

UI Design Mockups

Task gamification system — Design-I Project

UI 1 Dashboard — stats, today's tasks, streak & XP progress

UI 2 Leaderboard — weekly rankings, podium, and XP comparison

UI 3 XP History — filterable event log with multiplier breakdown

---

Design notes

- Consistent purple accent (#534AB7) used across all screens for interactive elements and XP indicators.
- All screens share the same top navigation bar with QuestLog branding and user avatar.
- Background: white cards on a light gray surface; font hierarchy uses 4 sizes.
- XP values are always highlighted in purple; status badges use semantic green / amber / red.

QuestLog Dashboard My Tasks Leaderboard Settings Alex M. [AM]

UI 1 — Dashboard

Total XP	Current level	Daily streak	Tasks done
4,820	Level 7	12 days	38
+120 today	Journeyman	1.5× multiplier	this week

Today's tasks

+ New task

- ✓ **Read Chapter 4 — System Design** — Completed 9:14 am — **+50 XP earned**
- ■ **Write lab report — Experiment 3** — Due today, 11:59 pm — **+80 XP**
- ■ **Submit CS project milestone 2** — Overdue — Apr 13 — **+120 XP**
- ■ **Review pull request — teammate branch** — Due Apr 16 — **+40 XP**

Level progress

Level 7 Level 8

4,820 / 7,500 XP — 2,680 to go

Weekly streak

M T W T [F] S S

Grace period active on Friday

Active XP rules

Streak bonus 1.5×

On-time completion 1.2× | Weekly goal met +200 XP

UI 2 — Leaderboard

Week of Apr 14, 2026 — resets every Monday

This week | This month | All time | Friends only

Top 3 this week

2nd
JL
Jordan L.
+680 XP

1st ■
SR
Sam R.
+940 XP

3rd
KP
Kai P.
+580 XP

#5 — Alex M. (you)

To reach #4
+45 XP needed

#	User	Level	XP this week	Tasks
1	**Sam R.**	Level 9 · Scholar	**+940**	11
2	**Jordan L.**	Level 8 · Expert	**+680**	9
3	**Kai P.**	Level 7 · Journeyman	**+580**	7
4	**Morgan T.**	Level 7 · Journeyman	**+785**	8
5	**Alex M. (you)**	Level 7 · Journeyman	**+740**	8
6	**Riley N.**	Level 6 · Apprentice	**+510**	6
7	**Casey D.**	Level 5 · Novice	**+390**	5

UI 3 — XP History

This week	This month	All-time total
+740 XP	**+3,210 XP**	**4,820 XP**

All | Completed tasks | Bonuses | Streak rewards | Last 7 days ■

Today — April 14, 2026

✓ **Read Chapter 4 — System Design**
Completed 9:14 am
50 base XP | 1.5× streak | 1.2× on-time
+90 XP uncapped

✓ **Weekly goal milestone reached**
Automatic bonus — 5 tasks completed this week
Weekly challenge rule
+200 XP bonus

✓ **Submit CS project milestone 1**
Completed 11:47 pm
100 base XP | 1.5× streak | cap applied: 150 XP
+150 XP capped

✓ **Write discussion post — Ethics in AI**
Completed 3:22 pm
60 base XP | 1.5× streak
+90 XP uncapped

QuestLog — Algorithm Design with Pseudo-Code

Design-II Project | 2026 Spring

This document presents the algorithm design for the three primary use cases of the QuestLog task-gamification system. Each algorithm corresponds to a UI screen described in the UI Design document. All algorithms use the same notation and layout.

Use Case 1: Mark Task as Complete (Dashboard UI)

When a user clicks the completion checkbox on a task in the Dashboard, the system validates the task state, computes the final XP reward by applying active multipliers, records an XP event, and updates the user's level and streak.

Algorithm 1: The *completeTask()* function — triggered when a user marks a task as done

```
Input:   task — the Task object selected by the user; user — the current User object   */
Output:  Updated user XP and level; a new XPEvent record; error message if invalid   */
1.1  if task.isCompleted then
1.2      display error: "Task is already completed" return
1.3  if task.dueDate is not null and task.dueDate < currentDateTime() then
1.4      isLate ← True
1.5  else
1.6      isLate ← False
1.7  if task.isEligibleForXP = False then
1.8      task.markComplete(currentDateTime()) return
1.9  baseXP ← task.getBaseXP()
1.10 activeRules ← fetchActiveXPRules(task.xpRuleId)
1.11 multipliers ← fetchApplicableMultipliers(user, isLate)
1.12 finalXP ← XPCalculator.calculate(baseXP, multipliers)
1.13 event ← new XPEvent(user.userId, task.taskId, baseXP, finalXP)
1.14 foreach m in multipliers do
1.15     record XPEventMultiplier(event.id, m.id, m.value)
1.16 user.totalXP ← user.totalXP + finalXP
1.17 user.recalculateLevel()
1.18 task.markComplete(currentDateTime())
```

1.19	*updateStreak*(*user*, *currentDate()*)
1.20	display: "Task complete! You earned +*finalXP* XP"

Use Case 2: View Leaderboard (Leaderboard UI)

When a user navigates to the Leaderboard page or changes the time-range or scope filter, the system fetches the ranked list of users by XP earned in the selected window, determines the current user's rank and gap to the next position, and renders both the podium and the full ranked list.

Algorithm 2: The *loadLeaderboard()* function — loads and ranks users for the Leaderboard UI

Input:	*userId* — the ID of the logged-in user; *timeRange* — selected time window ("weekly", "monthly", or "allTime"); *scope* — filter scope ("global" or "friends") */*
Output:	A ranked list of *UserRankEntry* records rendered in the leaderboard view; podium data for the top 3; current user's rank and XP gap to the next position */*
2.1	**if** *timeRange* = "weekly" **then**
2.2	*startDate* ← *currentDate()* − 7 days
2.3	**else if** *timeRange* = "monthly" **then**
2.4	*startDate* ← *currentDate()* − 30 days
2.5	**else**
2.6	*startDate* ← null */* no date filter — all-time */*
2.7	**if** *scope* = "friends" **then**
2.8	*userPool* ← *fetchFriendIds*(*userId*)
2.9	**else**
2.10	*userPool* ← *fetchAllUserIds()*
2.11	*entries* ← *fetchXPTotals*(*userPool*, *startDate*)
2.12	**if** *entries* is empty **then**
2.13	display: "No leaderboard data for this filter." **return**
2.14	*entries* ← *sortDescending*(*entries*, key: *xpTotal*)
2.15	*rank* ← 1
2.16	**foreach** *e* **in** *entries* **do**
2.17	*e.rank* ← *rank*
2.18	*rank* ← *rank* + 1

```
myEntry ← find entry in entries where entry.userId = userId
if myEntry.rank > 1 then
    nextEntry ← entries[myEntry.rank – 2]                    /* 0-indexed */
    myEntry.xpGap ← nextEntry.xpTotal – myEntry.xpTotal
else
    myEntry.xpGap ← 0                                        /* already in 1st place */
podium ← entries[0..2]                                       /* top 3 entries */
render podium: podium[0], podium[1], podium[2]
render my-rank banner: myEntry.rank, myEntry.xpTotal, myEntry.xpGap
render ranked list: entries sorted by rank ascending
```

Use Case 3: View XP History with Filters (XP History UI)

When a user navigates to the XP History page or changes the filter/time-range controls, the system fetches the matching XP events, builds per-event multiplier breakdowns, accumulates summary totals, and renders both the stat cards and the event list.

Algorithm 3: The *loadXPHistory()* function — loads and filters the XP event log for the current user

	Input: *userId* — the ID of the logged-in user; *filter* — selected filter category (e.g., "all", "completed", "bonuses"); *timeRange* — selected time window (e.g., 7, 30, or all) */*
	Output: A filtered, sorted list of *XPEvent* records rendered in the history view; summary totals displayed in stat cards */*
3.1	**if** *timeRange* = 7 **then** *startDate* ← *currentDate()* – 7 days
3.2	**else if** *timeRange* = 30 **then** *startDate* ← *currentDate()* – 30 days
3.3	**else** *startDate* ← null */* no date filter */*
3.4	*events* ← *fetchXPEvents*(*userId*, *startDate* ← *startDate*)
3.5	**if** *filter* = "completed" **then**
3.6	*events* ← filter *events* where *event.reason* = "task_completion"
3.7	**else if** *filter* = "bonuses" **then**
3.8	*events* ← filter *events* where *event.reason* = "bonus"
3.9	**else if** *filter* = "streak" **then**
3.10	*events* ← filter *events* where *event.reason* = "streak_reward"
3.11	**if** *events* is empty **then**
3.12	display: "No XP events found for this filter and time range." **return**
3.13	*weeklyTotal* ← 0; *monthlyTotal* ← 0; *allTimeTotal* ← 0
3.14	**foreach** *e* **in** *events* **do**
3.15	*multiplierList* ← *fetchMultipliersForEvent*(*e.eventId*)
3.16	*e.breakdown* ← *buildBreakdown*(*e.baseXP*, *multiplierList*)
3.17	*allTimeTotal* ← *allTimeTotal* + *e.finalXP*

3.18	**if** *e.awardedAt* ≥ *currentDate()* − 7 days **then**
3.19	*weeklyTotal* ← *weeklyTotal* + *e.finalXP*
3.20	**if** *e.awardedAt* ≥ *currentDate()* − 30 days **then**
3.21	*monthlyTotal* ← *monthlyTotal* + *e.finalXP*
3.22	render stat cards: *weeklyTotal*, *monthlyTotal*, *allTimeTotal*
3.23	render event list: sorted by *e.awardedAt* descending

Supporting Algorithm: Update User Streak

Algorithm 4 is a helper called by Algorithm 1 (step 1.19). It encapsulates the streak update logic including the grace-period rule, so that it can be reused whenever a user activity is recorded.

Algorithm 4: The *updateStreak()* function — updates the user's streak after a task is completed

	Input: *user* — the current User object; *today* — the current Date */
	Output: Updated Streak record (*dailyCount*, *weeklyCount*, *lastActivityDate*, *graceUsed*) */
4.1	*streak* ← *fetchStreak*(*user.userId*)
4.2	*last* ← *streak.lastActivityDate*
4.3	**if** *last* = *today* **then**
4.4	**return** /* *already updated today, no change* */
4.5	**if** *last* = *today* – 1 day **then**
4.6	*streak.dailyCount* ← *streak.dailyCount* + 1
4.7	*streak.graceUsed* ← False
4.8	**else if** *last* = *today* – 2 days **and** *streak.graceUsed* = False **then**
4.9	/* *grace period: user missed one day but has not used grace yet* */
4.10	*streak.dailyCount* ← *streak.dailyCount* + 1
4.11	*streak.graceUsed* ← True
4.12	**else**
4.13	*streak.dailyCount* ← 1 /* *streak broken, restart* */
4.14	*streak.graceUsed* ← False
4.15	*streak.weeklyCount* ← *streak.weeklyCount* + 1
4.16	*streak.lastActivityDate* ← *today*
4.17	persist *streak* to database

CS3704 - Design II - Achievements and Rewards

1 Introduction

This module is responsible for tracking user progress toward achievements, unlocking achievements when thresholds are met, awarding virtual currency (VC) as a reward, and allowing users to spend that currency in a virtual store.

2 Algorithm Design

2.1 UC-AR-1: Track Task Completion Progress

This use case is happens each time a user completes a task, such as checking off a to-do item or logging a habit. The system must identify which active achievements are relevant to the completed task type, increment their progress counters, persist the changes, and update the UI. If an achievement reaches its completion threshold as a result, the unlock process is triggered.

The logic is split across two algorithms: a helper that filters achievements by task type, and the main function that processes the filtered results.

Algorithm 1: The **getMatchingAchievements()** helper function

```
Input: achievements—full list of the user's achievements, taskType—the type
       string of the completed task
Output: Filtered list of achievements that are active and match the task type
matching ← empty list;
/* 1. Examine each achievement and retain only eligible ones */;
foreach achievement a in achievements do
    /* 2. Include only achievements that are still active and criteria-matched */;
    if a.status ≠ "Completed" and a.criteriaType = taskType then
        add a to matching;
    end
end
return matching;
```

Description: This helper iterates over all user achievements and returns only those that are still in progress and whose criteria match the type of the completed task.

Algorithm 2: The **onTaskCompleted()** function

```
Input: userID—the ID of the logged-in user, task—the completed task object
       containing its type
Output: Updated achievement progress counters; unlock triggered if any threshold
        is met
/* 1. Load all achievements and filter to those relevant to this task */;
achievements ← loadUserAchievements(userID);
matching ← getMatchingAchievements(achievements, task.type);
/* 2. Update progress for each matching achievement */;
foreach achievement a in matching do
    a.progressCount ← a.progressCount + 1;
    persistProgress(userID, a);
    /* 3. Reflect the updated counter in the UI progress bar */;
    updateProgressBarUI(a);
    /* 4. Trigger unlock if the threshold has now been reached */;
    if a.progressCount ≥ a.threshold then
        onAchievementUnlocked(userID, a);
    end
end
```

Description: This function coordinates the full task completion response. It delegates filtering to the helper, then increments and persists the progress counter for each relevant achievement. The unlock process is triggered automatically when a counter meets or exceeds its threshold.

2.2 UC-AR-2: Unlock Achievement

This use case is triggered when a progress counter reaches its threshold. The system must mark the achievement as completed exactly once, record the unlock timestamp, notify the user, and initiate the virtual currency award.

Algorithm 3: The **onAchievementUnlocked()** function

```
Input: userID—the ID of the user, achievement—the achievement object whose
       threshold was just met
Output: Achievement marked "Completed"; timestamp recorded; user notified; VC
        award triggered
/* 1. Guard against duplicate awards if already completed */;
if achievement.status = "Completed" then
  | return;
end
/* 2. Mark the achievement as completed and record the unlock timestamp */;
achievement.status ← "Completed";
achievement.unlockedAt ← getCurrentTimestamp();
persistAchievement(userID, achievement);
/* 3. Notify the user immediately if online, or queue for next login */;
if isOnline(userID) then
  | showUnlockModal(achievement);
else
  | queueNotification(userID, achievement);
end
/* 4. Trigger virtual currency reward for this achievement */;
awardVirtualCurrency(userID, achievement.rewardEventType);
```

Description: This function finalizes an achievement unlock. If the achievement is already completed, the function exits immediately and no further action is taken to prevent duplicate unlocks. Otherwise, the status and timestamp are transferred to the database. The online check determines whether the unlock is displayed now or queued for the next session. Finally, the reward process is triggered by passing the achievement's configured event type to **awardVirtualCurrency()**.

2.3 UC-AR-3: Award Virtual Currency

This use case is triggered at the end of UC-AR-2, but is designed generically enough to be called by any reward-triggering event in the system. The function looks up the correct reward amount, attempts an atomic balance update to prevent race conditions, retries up to three times on database failure, and notifies the user on success.

Algorithm 4: The **awardVirtualCurrency()** function

```
Input: userID—the ID of the user, eventType—the reward-triggering event type
       string
Output: User VC balance incremented; transaction logged; user notified
/* 1. Look up the configured reward amount for this event type */;
rewardAmount ← lookupRewardTable(eventType);
if rewardAmount = NULL or rewardAmount = 0 then
    return;
    /* No reward defined for this event; exit silently */;
end
attempts ← 0;
success ← False;
/* 2. Attempt atomic balance update with up to 3 retries on failure */;
while attempts < 3 and success = False do
    attempts ← attempts + 1;
    result ← atomicAddToBalance(userID, rewardAmount);
    if result = SUCCESS then
        /* 3. On success, log the transaction and notify the user */;
        logTransaction(userID, eventType, rewardAmount,
          getCurrentTimestamp());
        showCurrencyNotification(userID, rewardAmount);
        success ← True;
    else
        pause briefly before next retry;
    end
end
/* 4. If all retries exhausted, escalate the failure to an administrator */;
if success = False then
    logError(userID, eventType);
    alertAdmin(userID, eventType);
end
```

Description: The atomic balance update ensures no race condition can occur if two events fire simultaneously for the same user. The retry loop with a cap of three attempts stops indefinite blocking whiile allowing for some retries. If all retries fail, the error is logged and escalated.

2.4 UC-AR-4: Purchase Item with Virtual Currency

This use case is initiated when the user clicks the "Buy" button on an item in the virtual store. The logic is split into a validation helper, an execution helper, and a main coordinating function.

Algorithm 5: The **validatePurchase()** helper function

```
Input: userID—the ID of the logged-in user, item—the store item object to be
       validated
Output: True if all purchase conditions are satisfied, False otherwise
/* 1. Confirm the item exists in the store catalogue */;
if item = NULL then
    showError("Item not found.");
    return False;
end
/* 2. Confirm the user does not already own this item */;
if userOwnsItem(userID, item.id) then
    showError("You already own this item.");
    return False;
end
/* 3. Confirm the user has a sufficient VC balance */;
balance ← getVCBalance(userID);
if balance < item.price then
    showError("Insufficient funds.");
    return False;
end
return True;
```

Description: This helper performs sequential sanity checks before any transaction is attempted. Each failed check surfaces an error message to the user and returns False immediately, preventing unnecessary further checks or operations.

Algorithm 6: The **executePurchase()** helper function

```
Input: userID—the ID of the logged-in user, item—the validated store item to be
       purchased
Output: VC balance deducted; item added to inventory; transaction logged
/* 1. Deduct the item price from the user's current VC balance */;
balance ← getVCBalance(userID);
updateVCBalance(userID, balance − item.price);
/* 2. Grant the item and record the full transaction */;
addItemToInventory(userID, item.id);
logPurchaseTransaction(userID, item.id, item.price, getCurrentTimestamp());
```

Description: This helper is called after all validation and user confirmation have passed. It performs the balance deduction, inventory update, and transaction logging.

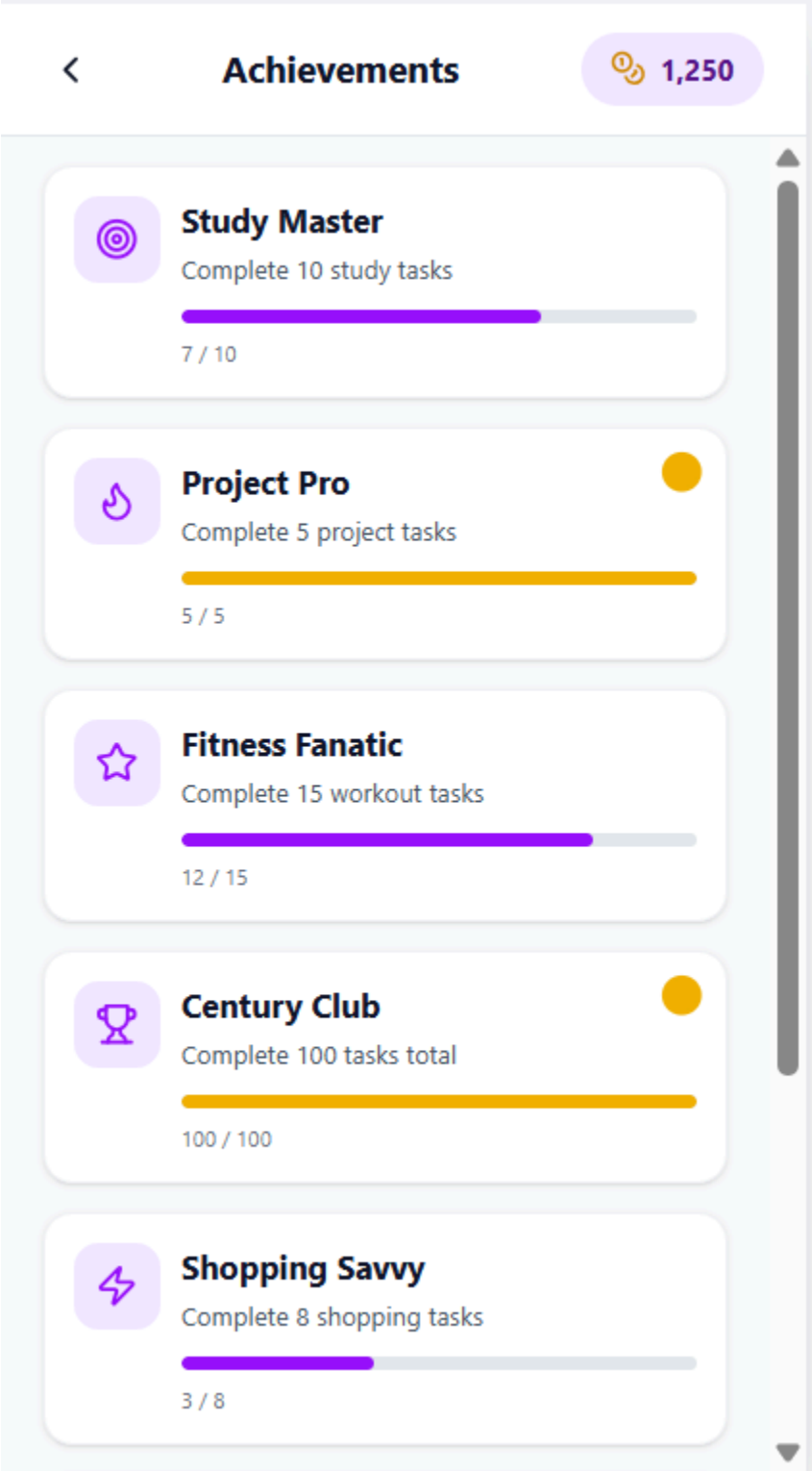
Algorithm 7: The **onPurchaseClicked()** function

```
Input: userID—the ID of the logged-in user, itemID—the catalogue ID of the
       selected store item
Output: Item purchased and applied to the user's account
/* 1. Retrieve the item from the store catalogue */;
item ← getItemFromStore(itemID);
/* 2. Run all sanity checks before proceeding */;
if validatePurchase(userID, item) = False then
    return;
end
/* 3. Require explicit user confirmation before committing the transaction */;
confirmed ← showConfirmationDialog(item.name, item.price);
if confirmed = False then
    return;
    /* User cancelled; no changes made */;
end
/* 4. Execute the transaction and confirm success to the user */;
executePurchase(userID, item);
showSuccessMessage(item.name);
promptEquipOrApply(item);
```

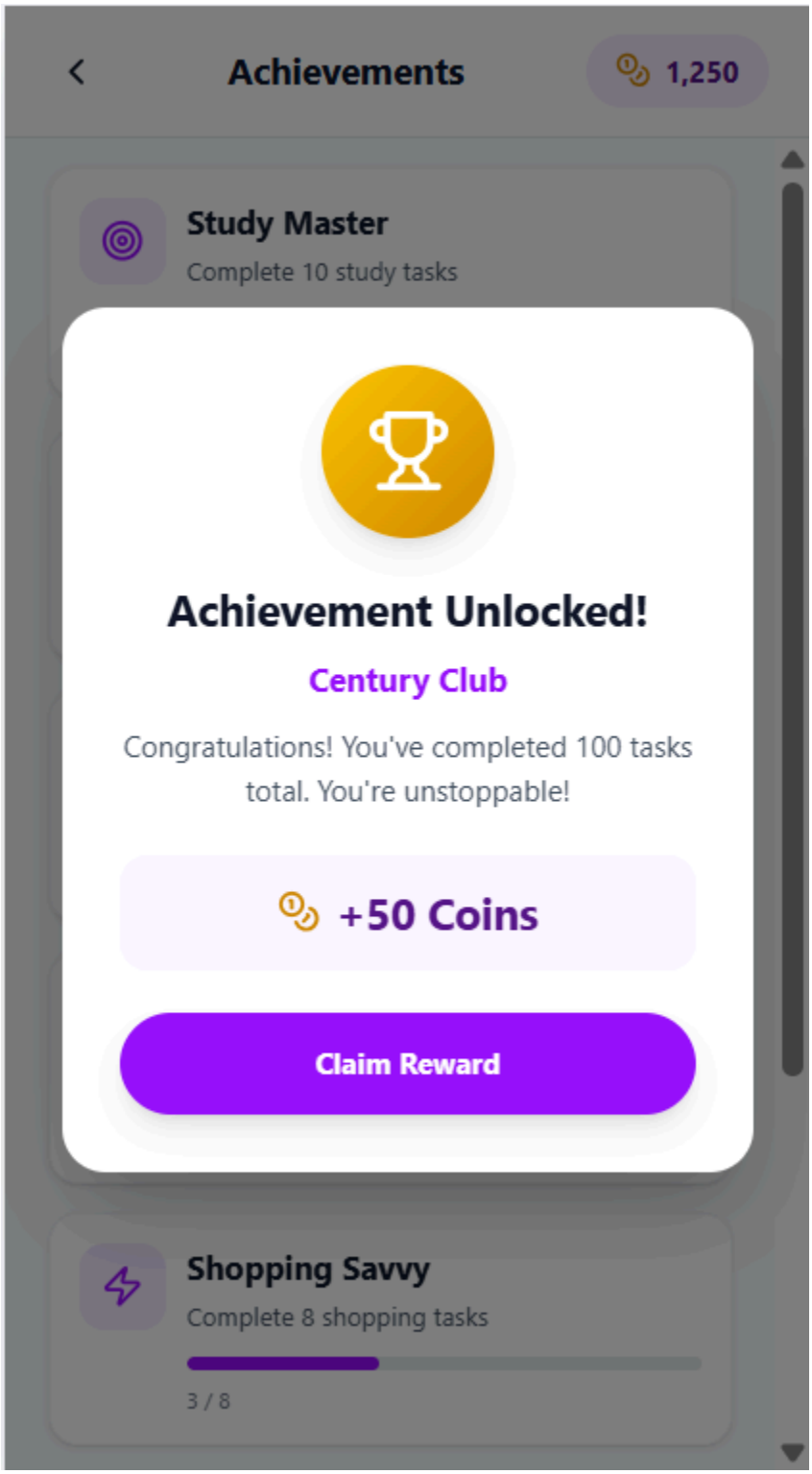
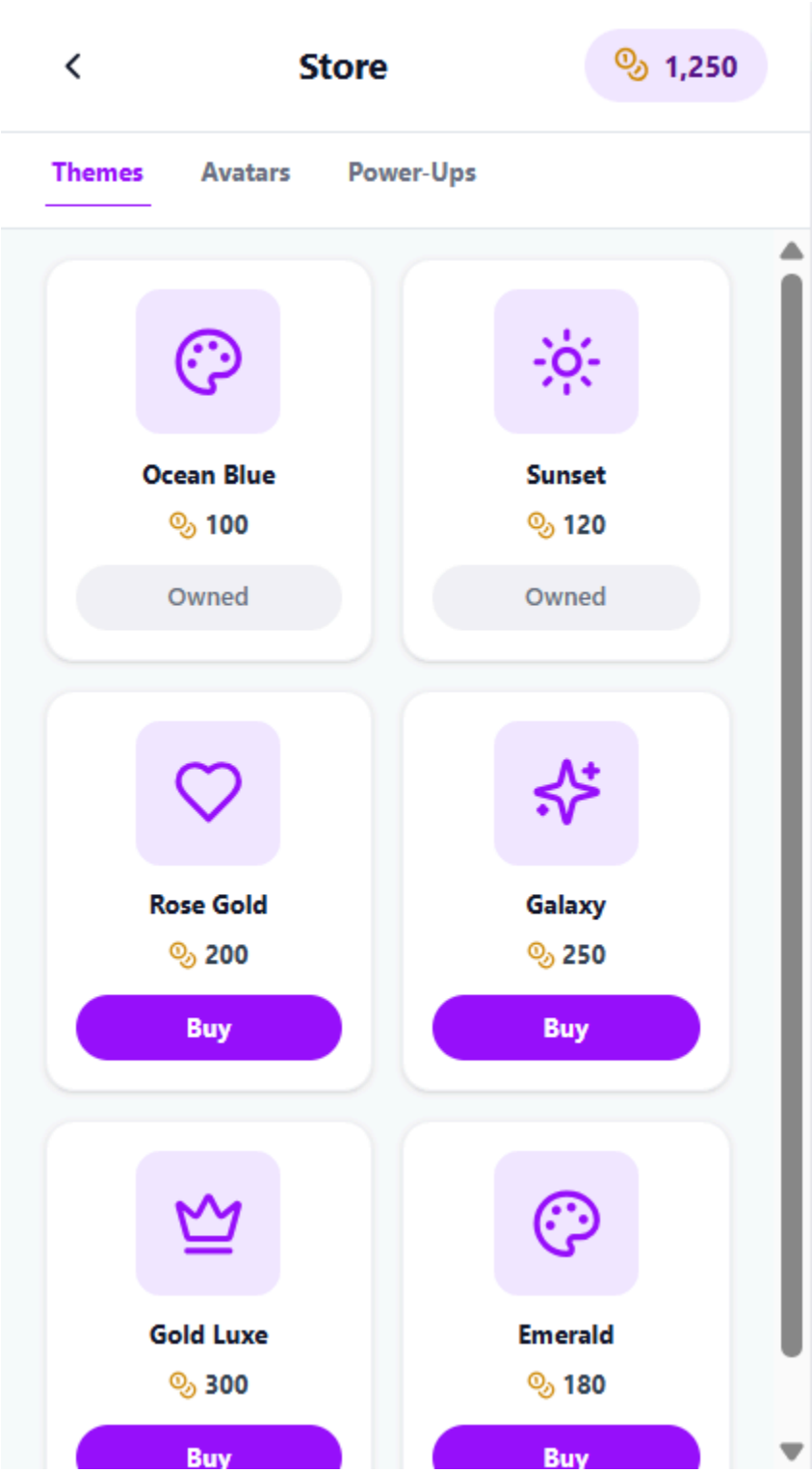
Description: This function coordinates the full purchase. It delegates all validation to **validatePurchase()** and all data modification to **executePurchase()**. The confirmation dialog gives the user a final chance to cancel before any changes are committed.

Mock UIs

UC-AR-1 – Track Task Completion Progress & UC-AR-2 – Unlock Achievement



UC-AR-3 – Award Virtual Currency



UC-AR-4 – Purchase Item with Virtual Currency